Taller de desarrollo web

ComCom

Departamento de Computación
FCEyN UBA

Primer Cuatrimestre 2026

Introducción

¿Qué constituye un sitio web?

- Una interfaz gráfica
- ¿Nada más?

Introducción

Front-end

Cuando nos referimos al desarrollo "front-end", estamos hablando de el trabajo de programar la interfaz que van a ver los usuarios cuando entran al sitio. Eso podría también podría incluir hacer el diseño, pero no necesariamente.
Lo que se arma en esta parte podría entenderse como un "cliente", un programa que corre en la máquina del usuario y que se comunica con el servidor solo cuando sea necesario a través de cierta "API".
Este taller va a ser una introducción a front-end.

Introducción

¿Servidor? ¿API? Back-end

Cuando un sitio se vuelve relativamente complejo, no puede correr o almacenarse entero en la máquina del usuario. No solo por un tema de recursos, sino que también por seguridad. Por ejemplo, en un sitio con autenticación, sería super inseguro y pesado almacenar todas las cuentas y contraseñas en las máquinas de los usuarios. Aparte, el sitio mismo tiene que estar subido a algún lado para que la gente pueda acceder.
Todo ese trabajo entra en el campo del back-end. Servidores, bases de datos, redes, entre otras cosas, son todas parte de nuestro trabajo como desarrolladores de software. Dependiendo del tamaño y objetivo del sitio, suele ser la mayor parte del desarrollo.

Introducción

En este taller no nos vamos a meter con back-end, pero si les interesa quédense atentes al taller de APIs! También la mayoría de los temas relacionados se ven con muchísima profundidad a lo largo de la carrera de computación.

Volviendo al front...

¿Y qué significa "desarrollar front"? ¿Cómo se programan estos clientes? ¿Qué lenguaje y herramientas usamos?
Los sitios web corren en navegadores web. Estos son Chrome, Firefox, etc. Estos navegadores entienden tres lenguajes:

- **HTML**: Define la estructura del sitio, su contenido*. En principio esta es la parte más importante, todo lo demás depende e interactúa con esta información.
- **CSS**: Define como se ve el sitio, su estilo. Tiene muchísimas opciones, lo cual lo hace bastante difícil de dominar, pero es necesario para hacer un sitio que no sea un texto negro sobre fondo blanco.
- **JavaScript**: A diferencia de los anteriores, este es un lenguaje de programación. Como tal, se ocupa de la lógica e interactividad del sitio. Es la parte más compleja de muchos de los sitios modernos y eleva las posibilidades a otro nivel.

Volviendo al front...

En este taller vamos a ver solo HTML y CSS. Por separado ya son super potentes y explicar JavaScript implicaría enseñar a programar, para lo cual existe la carrera.
También, como veremos más adelante, muchas veces no se programa en estos lenguajes de forma directa. Pero nos estamos adelantando, arranquemos desde el principio.

Hypertext Markup Language (HTML)

HTML es un lenguaje para armar documentos. "Hypertext" se refiere a los links que conectan los distintos sitios y documentos. "Markup" se refiere a que se usa una sintaxis particular para marcar cómo se debería estructurar el contenido.

La sintaxis de HTML consiste en el uso de *tags*. Hay alrededor de 140 de estas tags, pero lo normal es usar un subconjunto bastante más chico de ellas.

Las tags tienen el formato `<tag atributos>Contenido</tag>` o `<tag atributos/>` para elementos sin contenido. Dicho contenido podría ser texto o incluso otras tags. Veamos cuáles son las tags más importantes.

Estructura principal

Las tags más importante que cualquier sitio debe tener son `html`, `head` y `body`. La primera es la que define los límites del documento, sin esa tag no existe nada. La segunda es para definir información relevante al sitio pero que no va a ser visible para el usuario, eso incluye metadatos y un poco de información extra para que el navegador sepa como interpretar el sitio. Por último, `body` es donde va el contenido del sitio, básicamente todo.
Cualquier sitio normal sigue el siguiente esquema:

```
1 <html>
2     <head>
3         Metadatos, estilo, etc...
4     </head>
5     <body>
6         Contenido
7     </body>
8 </html>
9
```

Contenidos de `head`, metadatos

- `title` se usa para elegir el título de la página. Dicho título es el que se ve en la pestaña correspondiente del navegador.
- `link` define relaciones con recursos externos al documento. Se usa principalmente para elegir la hoja de estilos (CSS) pero también otras cosas como el ícono del sitio (suele aparecer al lado del título). En el caso más general usa el atributo `href` para definir la dirección del recurso y `rel` para especificar cual es dicha relación.
- `style` permite agregar estilo directamente ahí pero no es muy común hacer eso.
- `meta` sirve para agregar cualquier otro tipo de información que no se pueda representar con las tags anteriores.

Estructura principal pero menos principal

HTML viene con los elementos `header`, `main` y `footer` para organizar el sitio en esas tres partes. También incluye cosas como `aside` para cosas que justamente deberían ir a un costado, `article` para artículos, etc. (todas bastante auto explicativas).
Sin embargo, suele pasar (y es discutible si es algo bueno o malo) que el único elemento que se termina usando para organizar el sitio es `div`. El elemento justamente es un contenedor genérico que no representa nada por si mismo.
Todos estos elementos tienen formato "block", que, ahora mismo y sin entrar en detalles, significa que ocupan el ancho entero del sitio. También implica otras cosas pero todo eso va a importar más cuando nos pongamos con los estilos.

Texto

Los elementos `hN`, donde la *N* va de 1 a 6, son encabezados. La importancia va de menor a mayor, donde `h1` es el título principal del documento (y debería haber solo uno) y `h6` es un título bastante menor.
El elemento `p` representa un párrafo, aunque se puede meter todo el texto que quieran ahí adentro.
Esos son los principales elementos que contienen texto y ambos tienen formato block. Pero también hay elementos que modifican el texto que está adentro de los `p` y `h`.
`a` es para agregar links, `strong` es para marcar importancia, `em` es para énfasis, etc. Al igual que `div`, la versión genérica de los modificadores es `span`. Todos estos elementos tienen formato "inline", lo cual por ahora solo implica que van en el medio del contenido.

Listas y tablas

También tenemos listas y tablas.

- `ul` es para listas no ordenadas (el punteado no tiene números) y `ol` es para listas ordenadas (el punteado si tiene números (o letras (o lo que quieran (se puede cambiar todo)))). Los elementos que van adentro de estas listas deberían ser `li`.
- `table` es para tablas. Hay un montón de elementos relacionados a las tablas para organizarlas como quieran, pero los más importantes son `tr` para filas, `th` para cabezales de columnas y `td` para celdas.

Imágenes y formularios

Para terminar con los elementos más comunes tenemos imágenes y formularios.

- Para usar imágenes está `img`. El atributo `src` define la dirección de la imagen a mostrar. Tiene unos cuantos atributos más para cambiar como aparece y tocar temas de accesibilidad.
- Los formularios son con `form` y los campos de dichos formularios son con `input`. El atributo `type` define que tipo de campo es.

Comentarios

También podemos escribir comentarios con una sintaxis muy similar:

```
<!— Esto es un comentario —>

<!—
    Todo lo que esté entre esas tags no es leído por el navegador
    <p>No importa que haya adentro</p>
—>
```

Ejercicio - Perfil

En este taller cada los ejercicios están divididos en distintos archivos .html y .css. En la carpeta "ejercicios" están los archivos a completar y en "resueltos" los archivos completos. La idea es que solo miren los resueltos al final para comparar con lo que hicieron o que a lo sumo los abran (en el navegador, no un editor de texto) para ver cómo podrían hacer que se vea el resultado. En los archivos a completar se pueden ver las consignas comentadas.

Este primer ejercicio es solo de html y se puede resolver usando las tags que vimos hasta ahora. La idea es que nos cuenten un poco sobre ustedes y sobre la relacion que tienen con la carrera.

Estilo, CSS

Si entraron a cualquier sitio web de los últimos 20 años, seguramente hayan notado que se ven bastante más complejos que lo que hicimos hasta ahora con HTML.
Entra CSS, Cascading Style Sheets. El lenguaje existe para cambiar la forma en la que se ven los documentos HTML. Hasta ahora dependíamos de cómo los distintos elementos fueran renderizados por el navegador, pero ahora podemos cambiar eso completamente.

¿Quién necesita JS?

Como dato curioso, HTML y CSS juntos son turing complete. Eso significa que, técnicamente, se pueden usar para hacer cualquier cosa que los otros lenguajes de programación puedan hacer, incluyendo JavaScript. Esto no significa que eso sea una buena idea. Estos lenguajes no fueron pensados para eso y tratar de usarlos de esa forma sería increíblemente incómodo y no valdría la pena.
De todas formas, cuando alguien les diga que HTML y CSS no son lenguajes de programación, pueden responder con este dato.

Sintaxis de CSS

CSS funciona con *reglas*. Un archivo .css (o una tag `style` en un archivo .html) contiene tantas reglas como uno quiera, una atrás de otra, definiendo a qué partes del documento se le deberían aplicar qué propiedades. Estas reglas siguen el siguiente formato:

```
1 selector {
2     propiedad1: valor1;
3     propiedad2: valor2;
4     ...
5 }
6
```

El selector indica qué elementos van a ser afectados por la regla. Las propiedades indican cómo van a ser afectados.

Selectores, clases e ids

Los selectores más básicos son selectores de tipo, de clase y de id.
Un selector de tipo dice que la regla va a aplicar a todos los elementos del tipo indicado. Por ejemplo, si el selector es `p`, todos los elementos `p` van a tener la regla.
Los selectores de clase y de id funcionan igual, pero hay que marcarlos con un punto y con un numeral respectivamente. Por ejemplo, si el selector es `.contenedor`, todos los elementos con la clase "contenedor" van a tener esa regla. Lo mismo con `#titulo-principal` y el elemento con id "titulo-principal".

¿Clases e ids?

Pero, ¿qué son las clases y los ids?
Todo elemento de HTML puede tener los atributos `class` e `id`. El primero es una lista de nombres separados con espacios. El segundo es un solo nombre.
La idea de las clases es que sean indicadores de características compartidas por múltiples elementos a lo largo del documento. También, un elemento puede (y suele) tener múltiples clases. La diferencia con los ids es que el id debe ser único.
El motivo por el que estos atributos existen es para poder identificar ciertos elementos o conjuntos de elementos del sitio. Como ya vimos, en CSS esto nos deja aplicar reglas de forma mucho más específica. En el caso de JS, nos permitiría encontrar e interactuar elementos de la misma forma.

Ejemplo de HTML y CSS usando clases e ids

```html
<div class="contenedor" id="contenedor-principal">
    <h1 id="titulo-principal">Buenas tardes<h1>
</div>
<div class="contenedor azul">...</div>
```

```css
.contenedor {
    display: flex;
}
#contenedor-principal {
    background-color: orange;
}
#titulo-principal {
    color: red;
}
.azul {
    background-color: blue;
}
```

Más selectores

Pero esos tres no son los únicos tipos de selectores. Se pueden combinar entre sí de distintas formas para armar selectores mucho más complejos que nos permiten ser mucho más específicos sobre a qué partes del sitio se les debería aplicar qué reglas.
Algunos ejemplos son:

- `A, B` para indicar que la regla debe aplicar a `A` y a `B`.
- `A B` para indicar que la regla debe aplicar a los `B` que sean descendientes de `A`.
- `A:PC` para indicar que la regla debe aplicar a `A` cuando tiene la pseudoclase `PS`.

Entre otros.

Valores para las propiedades

Ya hablamos de los selectores. De las propiedades no hay mucho que decir, hay un montón, cada una tiene un nombre único y ciertos valores que puede tomar. Pero esos valores no son ni un poquito sencillos.
Por ahora vamos a concentrarnos en dos tipos de valores posibles:

- *keyword*: Son valores con nombres específicos y hacen cosas específicas dependiendo de para qué propiedad están siendo usados.
- Magnitudes: Son valores de la forma *Nu*, donde *u* es la unidad y *N* es la cantidad de dicha unidad. Estas unidades pueden ser absolutas o relativas. Las absolutas son siempre lo mismo, sin importar en donde aparezcan*. Las relativas representan distintas magnitudes dependiendo del contexto, ya sea dado por el elemento al que se está aplicando la regla o por el propio navegador en el que está viéndose el sitio.

Más adelante vamos a profundizar un poco sobre las unidades, pero por ahora algunas útiles e interesantes son `px`, `cm`, `%` y `em`.

Algunas propiedades básicas

Algunas propiedades básicas para que empecemos a tocar un poco el estilo de nuestros sitios son:

- `color` para cambiar el color del texto. El color puede escribirse por nombre, por código hexadecimal, o con una variable.
- `background-color` para cambiar el color del fondo. Recibe lo mismo que `color`.
- `border` para ponerle un borde al elemento. Recibe un grosor, un estilo y un color.
- `font-size` para cambiar el tamaño del texto. Recibe una magnitud.
- `font-family` para cambiar la fuente del texto. Recibe el nombre de la fuente (o familia de fuentes).
- `background-image` para poner una imagen de fondo. Recibe una dirección a la imagen o algunos otros valores medio raros.
- `text-align` para alinear el texto. Recibe como valor alguna keyword indicando el modo.

Ejercicio - Historia

En este ejercicio tienen que retocar y arreglar el estilo del documento sobre la historia de nuestra carrera. Pueden usar las propiedades que vimos en la diapositiva anterior y también pueden buscar otras propiedades que les interesen en internet.
Este es un buen momento para mencionar developer.mozilla.org, lugar en el que está documentado casi todo lo relacionado a HTML, CSS y JS. El sitio tiene buenas explicaciones, tutoriales, ejemplos y ejercicios. Está buenísimo. Pueden buscar ahí lo que quieran para resolver los ejercicios del taller.
Eso si, por ahora eviten meterse con reglas que cambien el "layout" del sitio. Eso incluye padding, márgenes, posicionamiento, display, etc.
Después de este ejercicio nos vamos a meter con esos temas.

El modelo de caja

Hasta ahora solo cambiamos cómo se ven los elementos en el sitio. Sin embargo, una de las cosas más potentes de CSS es que también podemos cambiar cómo se organizan. Vamos a cambiar el *layout* del sitio.
En CSS, todos los elementos están en cajas. Estas cajas tienen 4 partes:

- El contenido: Lo que metemos en los elementos. Texto, imágenes, otros elementos, etc.
- *Padding*: Justamente es un relleno alrededor del contenido. Suele verse como parte del elemento.
- Borde: El espacio entre el padding y el margen. Marca el límite del elemento.
- Margen: El espacio entre el elemento y otros elementos. En general no se ve como parte del elemento. Cuando hay dos elementos juntos sus márgenes se suman.

En ese orden desde adentro hacia afuera.

El modelo de caja

Por defecto, el ancho y alto de un elemento se calculan solo en base al contenido, por lo que hay que sumar el padding, borde y margen para conseguir el tamaño total de la caja. La regla `box-sizing: border-box` permite cambiar esto para que se calcule en base al contenido, padding y borde, sin incluir el margen.

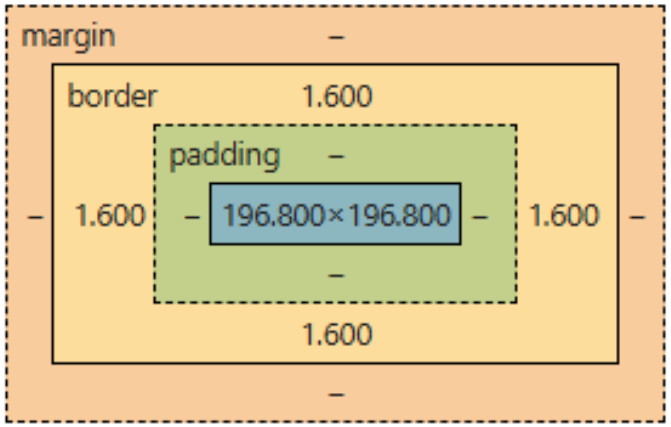

Figura: El modelo de caja

Manipulando la caja

Ahora que entendemos cómo están compuestos los elementos, empezemos a cambiarles la forma. El padding y el márgen tienen propiedades que permiten cambiarlos en cualquier dirección:

- `padding: valorParaLas4Direcciones`
- `padding: valorVertical valorHorizontal`
- `padding: arriba derecha abajo izquierda`

Las mismas versiones existen para `margin`. También existen propiedades específicas para cada lado (por ej. `padding-top`), no hace falta setear las cuatro direcciones siempre.
La propiedad del borde es un poco distinta, pero también tiene propiedades específicas que permiten tocar cada lado por separado.

- `border: grosor estilo color`

Manipulando la caja

También podemos cambiar el tamaño del contenido! O por lo menos intentarlo. Las propiedades `width` y `height` hacen exactamente eso. Como dijimos antes, si `box-sizing` es `border-box`, los valores de estas propiedades incluyen al padding y el borde.
Acá entramos un poco en terreno tormentoso. Si el tamaño que le damos al contenido no es suficiente, el contenido podría escaparse del contenedor, cosa que se conoce como *overflow*. Si el tamaño que le damos es demasiado, nos van a quedar agujeros medio raros.
También existen propiedades `min-width` y `max-width` (y lo mismo para height) que limitan los valores de las propiedades anteriormente mencionadas. Ahora, ¿por qué existen esas propiedades? ¿Por qué alguien pondría un valor de width mayor al de max-width?

Unidades y un flash de diseño receptivo (*responsive*)

Más atrás mencionamos muy al pasar algunas unidades que se usan para los valores de las propiedades. Estas unidades se distinguen en dos categorías:

- Absolutas: Indican la misma magnitud sin importar en qué contexto aparezcan*. Son cm, mm, Q ($\frac{1}{4}$mm), in, pc, pt y px.
- Relativas: La magnitud que representan depende del contexto. Más detalles en la siguiente diapo.

Volviendo a `width` y `max-width`, alguien podría definir el ancho de un elemento con una unidad relativa pero después ponerle un límite con una unidad absoulta.

Unidades relativas

Las unidades relativas son las siguientes:

- em se refiere al tamaño de la fuente del elemento al que se le aplica la regla*. `2em` es el doble de ese tamaño.
- rem se refiere al tamaño de la fuente del elemento raíz.
- vh se refiere a la altura del *viewport*, el rectángulo en el que se está mostrando el sitio. `1vh` es 1 % de ese tamaño. La versión para el ancho es vw. Ver las versiones con s, l y d.
- vmin la más chica entre vh y vw. vmax es la más grande.
- Los porcentajes se refieren a un porcentaje de la propiedad en el elemento padre. Por ejemplo: `width: 50 %` representa la mitad del ancho que el elemento padre.
- fr se usa para específicamente para grids.

Hay unas cuántas más mucho más específicas pero no son muy comunes.

Posición

Así como podemos cambiar la forma de los elementos, también podemos cambiar dónde se ubican. La propiedad `position` define la forma en la que los elementos se ubican en el sitio. Las propiedades `top`, `bottom`, `left` y `right` mueven al elemento de distintas formas según el valor de `position`. Dichos valores son:

- `static`: El valor por defecto.
- `relative`: El elemento se posiciona relativamente a la posición que le tocaría normalmente. Sigue afectando a otros elementos.
- `absolute`: El elemento se posiciona relativamente al ancestro posicionado más cercano. Si ningún ancestro fue posicionado entonces es relativo al viewport. Deja de afectar a otros elementos.
- `fixed`: Como absolute pero directamente es relativo al viewport.
- `sticky`: Funciona raro. Similar a relative, pero cuando quedaría afuera por scrollear, se pega al tope.

Flexbox y grids. La propiedad `display`.

La propiedad `position` se usa para acomodar elementos de forma muy específica. Muchas veces, lo que queremos es acomodar varios elementos entre sí y de acuerdo a la caja que los contiene. Ahí surgen las *grids* y las *flexible boxes*.

Una grid se usa para partir un contenedor en secciones rectangulares. Se puede partir en tantas secciones como uno quiera y pueden tener cualquier tamaño. Es como una especie de mondrian.

Una flexible box se usa para acomodar elementos de forma que cambien sus tamaños acorde a la cantidad de espacio que haga falta. Son tan útiles que suelen estar por todos lados.

Un elemento se pone en modo grid o flexbox usando la propiedad `display` con los valores `grid` y `flex` respectivamente. Hasta ahora no la habíamos mencionado, pero esa propiedad también puede definir si el elemento es `inline` o `block`. También se le puede dar el valor `none` para sacar el elemento del sitio.

Grid

Flexbox

Como el nombre lo indica, un contenedor flexbox trata a sus elementos hijos como cajas de espaciado (y a veces tamaño) adaptable al espacio disponible. El eje principal sobre el cual se posicionan se define con la propiedad `flex-direction`, que puede ser `row`, `column`, o cualquiera con `-reverse` al final.

Para el posicionamiento, desde el contenedor usamos `justify-content` para controlar posicion de elementos en el eje principal. Esto incluye el espacio entre elementos. Algunas posibilidades son:

- `start`, `end`, `center`: Elementos en la posición indicada y "pegados", separados solo por sus margenes
- `space-between`: Aleja los elementos lo más posible, usando todo el espacio
- `space-evenly`: Similar al anterior, dejando espacio también con los bordes

`align-items` hace lo mismo en el eje opuesto, sin poder definir espaciado que no tendría sentido.

Flexbox

Como alternativa a tener que esconder cosas en caso de overflow, podemos usar `flex-wrap`, definiendo que los elementos se posicionen en una segunda fila o columna si no entran, que permite automáticamente adaptarse a pantallas de distintos tamaños.
En este caso, align-content nos dice como alinear la segunda fila o columna. Funciona igual que el `justify-content` en el otro eje.
También es posible indicarles a los elementos que se estiren a ocupar cierta proporción del espacio que sobra usando `flex`.
Hay otras propiedades de flex que modifican otros detalles del posicionamiento, que siempre pueden buscar si les interesa.

Ejercicio -

Ejercicio -

Cheatsheet de flexbox
Cheatsheet de grid